1. Configuración Inicial y Repositorio Local

- **`git config --global user.name "Tu Nombre"`**: Configura la identidad del usuario para los commits.
- **`git config --global user.email "tu@email.com"`**: Configura el correo electrónico del usuario.
- **`git init`**: Inicializa una carpeta normal convirtiéndola en un repositorio de Git.

2. Flujo de Trabajo (Add, Commit y Log)

- **`git add <archivo>`** o **`git add .`**: Añade archivos al área de preparación (*staging area*).
- **`git commit -m "mensaje"`**: Guarda los cambios preparados en el historial local con un mensaje descriptivo.
- **`git commit -am "mensaje"`**: Combina la acción de `add` y `commit` exclusivamente para archivos que ya están bajo seguimiento (*tracked*).
- **`git log --oneline`**: Muestra el historial de commits en formato resumido de una línea.

3. Conexión Remota y GitHub

- **`git remote add origin <URL>`**: Vincula el repositorio local con el servidor remoto.
- **`git remote -v`**: Verifica las conexiones remotas configuradas.
- **`git push -u origin main`**: Sube los commits locales a la nube por primera vez y establece la relación permanente con la rama remota.
- **`gh auth login-with-token < auth.txt`**: Autentica la terminal de GitHub CLI usando un token almacenado en un archivo.
- **`gh auth setup-git`**: Configura la autenticación de Git mediante GitHub CLI.

4. Ramas (*Branches*) y Fusiones (*Merge*)

- **`git branch -a`**: Muestra todas las ramas disponibles (tanto locales como remotas).
- **`git branch <nombre-rama>`**: Crea una nueva rama local sin cambiar de posición.
- **`git checkout <nombre-rama>`** (o **`git switch`**): Cambia la copia de trabajo a la rama seleccionada.
- **`git checkout -b <nombre-rama>`**: Crea una nueva rama y salta a ella de forma inmediata en un solo paso.

- **`git merge <nombre-rama>`**: Fusiona los cambios de la rama indicada dentro de la rama en la que estés posicionado actualmente.

5. Etiquetas y Versiones (*Tags*)

- **`git tag`**: Lista todas las etiquetas creadas por orden alfabético.
- **`git tag <version>`**: Crea una etiqueta ligera apuntando al commit actual (ej. `git tag v1.0`).
- **`git push origin <version>`**: Sube una etiqueta específica a GitHub.
- **`git push origin --tags`**: Sube todas las etiquetas locales pendientes al servidor remoto.
- **`git tag -d <version>`**: Elimina la etiqueta de forma local.
- **`git push origin :refs/tags/<version>`**: Elimina la etiqueta del servidor remoto en GitHub.

6. Gestión de Crisis y Errores (*Reset* vs *Revert*)

- **`git reset --soft HEAD^`**: Deshace el último commit pero conserva los cambios intactos en el código local.
- **`git reset --hard HEAD^`**: Deshace el último commit y descarta totalmente los cambios de los archivos, volviendo al estado anterior.
- **`git revert <HASH>`**: Crea un nuevo commit seguro que deshace los cambios de un commit previo (obligatorio si ya se ha hecho `push`).

7. Automatización (*Make*)

- **`make <regla>`**: Ejecuta las instrucciones automatizadas definidas en un archivo `Makefile` (como compilar o ejecutar scripts)

name: <Nombre del Workflow>

1. Disparador: Cuándo se ejecuta el workflow

on:

  push:

    branches: [<nombre-de-la-rama>]

2. Trabajos (Jobs) que se ejecutarán (pueden correr en paralelo o secuencial)

jobs:

```
<nombre-primer-job>:
  runs-on: <sistema-operativo>  # Ej: windows-latest, ubuntu-latest, macos-latest
  steps:
    # Paso básico para descargar el código del repositorio
    - name: <Descripción del paso>
      uses: <nombre-de-la-accion-externa>  # Ej: actions/checkout@v6

    # Paso opcional para configurar entornos (ej. Python, Node, etc.)
    - name: <Descripción del paso>
      uses: <nombre-de-la-accion-externa>  # Ej: actions/setup-python@v6
      with:
        <parametro-configuracion>: <valor>  # Ej: python-version: '3.12'

    # Paso para ejecutar comandos de consola mediante la terminal
    - name: <Descripción del paso>
      run: <orden-o-comando-de-consola>  # Ej: python -m unittest -v

<nombre-segundo-job>:
  runs-on: <sistema-operativo>
  steps:
    - name: <Descripción del paso>
      uses: <accion-externa>

    - name: <Descripción del paso>
      run: <comando-de-consola>
```

🌿 Resumen Práctico: Ramas, Fusiones y Colaboración

1. Visualización y Creación de Ramas

- **`git branch -a`**: Muestra todas las ramas disponibles en el repositorio (tanto locales como remotas).
- **`git branch <nombre-rama>`**: Crea una nueva rama local, pero **no** te posiciona automáticamente en ella (te mantiene en la rama actual).
- **`git branch -M main`**: Cambia o fuerza el nombre de la rama actual a `main` (muy útil si por defecto se creó como `master`).

2. Navegación y Cambio de Rama

- **`git checkout <nombre-rama>`** (o **`git switch <nombre-rama>`**): Cambia la copia de trabajo a la rama seleccionada.
- **`git checkout -b <nombre-rama>`** (o **`git switch -c <nombre-rama>`**): ¡El **comando estrella**! Crea una nueva rama y salta a ella de forma inmediata en un solo paso.

3. Subir Ramas al Servidor Remoto (GitHub)

- **`git push -u origin <nombre-rama>`**: Sube una rama local creada por ti a GitHub por primera vez y establece la relación permanente de seguimiento.

4. Fusión de Ramas (`git merge`)

Integra los cambios de una rama independiente dentro de la rama en la que te encuentres posicionado. Los pasos obligatorios para hacer un *merge* sin errores son:

1. Asegúrate de estar en la rama receptora (ej. `main`): `git checkout main`.
2. Actualiza tu rama local con el servidor para evitar conflictos: `git pull origin main`.
3. Ejecuta el comando de fusión: `git merge <nombre-rama-a-fusionar>`.

5. Resolución de Conflictos de Fusión (Merge Conflict)

Ocurre cuando tú (o distintas personas) habéis modificado exactamente las mismas líneas de un archivo. Pasos para solucionarlo en el examen:

1. Git marcará las líneas conflictivas en el archivo con símbolos especiales (<<<<, ====, >>>>).
2. Abre el archivo, borra las marcas y deja únicamente el código definitivo que quieras conservar.

3. Prepara el archivo resuelto: `git add .`.
4. Finaliza la fusión guardando los cambios con un commit: `git commit -m "Resolver conflicto de fusión"`.